Membaca dan Mengembangkan Small LLM Cerita

Panduan kode, rumus, dan contoh langkah demi langkah

Dokumentasi proyek `train-llm-from-scratch`

23 Oktober 2026

Abstract

Dokumen ini menjelaskan Small LLM dalam proyek ini dari teks mentah sampai kalimat yang dihasilkan. Sasaran pembacanya adalah orang yang sudah mengenal Python dasar, tetapi baru mulai mempelajari model bahasa. Setiap rumus langsung dihubungkan dengan bagian kode dan diterangkan memakai contoh kecil. Implementasi lokal dan notebook Colab dibahas secara terpisah bila cara evaluasi atau perangkatnya berbeda. Dokumen ini tidak mengandaikan model sudah mampu menulis cerita yang baik.

Contents

1 Peta besar: apa yang sedang dipelajari?

Small LLM di sini adalah *decoder Transformer* yang belajar menebak **byte berikutnya**. Ia tidak menerima arti kata secara langsung. Jika potongan teks berbunyi "Pagi itu", tugasnya adalah memberi probabilitas untuk byte yang mungkin muncul sesudahnya. Selama training, tebakan itu dibandingkan dengan byte yang benar; parameter model lalu diubah sedikit.

Cerita ⟶ byte UTF-8 ⟶ jendela input dan target ⟶ embedding ⟶ blok Transformer ⟶ skor 257 token ⟶ loss ⟶ pembaruan bobot

Analogi. Bayangkan murid diminta melanjutkan cerita satu potong sangat kecil setiap kali. Tokenizer menyiapkan kartu-kartu potongan, attention mengizinkan murid melihat kartu sebelumnya, dan loss adalah nilai kesalahan dari guru. Optimizer mengubah kebiasaan murid berdasarkan kesalahan itu.

1.1 Istilah yang dipakai

Token Satu ID yang diprediksi model. Dalam proyek ini satu token biasanya satu byte; ID 256 berarti akhir cerita.

Parameter/bobot Angka yang dipelajari, misalnya isi matriks pada lapisan linear.

Embedding Vektor angka yang mewakili token atau posisinya.

Logit Skor mentah untuk setiap kemungkinan token. Logit belum menjadi probabilitas.

Loss Ukuran kesalahan prediksi. Lebih kecil pada data validasi berarti prediksi byte validasi lebih baik, selama datanya sama.

Gradien Arah perubahan loss terhadap setiap parameter.

Checkpoint Salinan bobot dan konfigurasi model untuk inferensi.

1.2 Peta berkas kode

Berkas	Tanggung jawab
`small_llm/tokenizer.py`	Mengubah teks ke byte, mengenali akhir cerita, dan mengubah byte kembali ke teks.
`small_llm/data.py`	Membaca satu cerita per berkas, membagi train dan validasi per cerita, membuat pasangan input–target.
`small_llm/config.py`	Membaca JSON, memeriksa nilai konfigurasi, memilih MPS atau CPU pada program lokal.
`small_llm/model.py`	Embedding, causal attention, blok Transformer, cross-entropy, dan generasi yang menjaga UTF-8 tetap valid.
`small_llm/sparse.py`	Lima pilihan MLP: dense, pruned, topk, moe, dan set.
`small_llm/train.py`	Training satu varian, evaluasi berkala, metrik, dan checkpoint.
`small_llm/compare.py`	Menjalankan beberapa varian dan membuat tabel perbandingan.
`small_llm/generate.py`	Memuat checkpoint dan melanjutkan prompt.

Berkas	Tanggung jawab
`small_llm/__init__.py`	Menandai `small_llm` sebagai paket Python; tidak menjalankan training.
`tests/test_small_llm.py`	Memeriksa sifat penting kode dengan model dan data kecil.
Notebook Colab	Versi mandiri dalam `small_llm_frog_storytelling_colab.ipynb`; seluruh kode ditulis di notebook, dengan dataset Hugging Face.

2 Langkah 1: data menjadi token

2.1 Tokenizer byte: `tokenizer.py`

UTF-8 mengubah karakter menjadi satu atau beberapa byte. Fungsi `encode(text)` menjalankan `text.encode("utf-8")` lalu mengubahnya menjadi daftar bilangan. Kosakatanya tetap:

$$\mathcal{V} = \{0, 1, \ldots, 255, 256\}, \qquad |\mathcal{V}| = 257.$$

ID 0–255 adalah nilai byte. ID 256 adalah *END_OF_STORY*, penanda akhir dokumen yang ditambahkan saat data dibaca. Contoh sederhana:

$$\text{"A"} \mapsto [65], \qquad \text{"AB"} \mapsto [65, 66], \qquad \text{"A" + akhir cerita} \mapsto [65, 256].$$

Huruf beraksen atau simbol tertentu bisa memerlukan lebih dari satu byte. Karena itu panjang konteks 256 berarti **256 byte/token**, bukan 256 kata. Fungsi `decode(ids)` berhenti pada ID 256. Jika daftar byte yang diberikan rusak, fungsi itu mengganti bagian UTF-8 yang tidak lengkap agar teks tetap dapat ditampilkan.

Analogi. Token seperti potongan puzzle. Satu huruf biasa dapat menjadi satu potong, sementara karakter lain perlu beberapa potong. Kartu 256 adalah tanda "cerita selesai".

Perhatikan. Tokenizer byte tidak mempunyai masalah kata "tidak dikenal", karena setiap teks UTF-8 dapat dipecah menjadi byte. Namun urutan token menjadi lebih panjang dibanding tokenizer subkata.

2.2 Pemisahan cerita: `data.py`

`load_stories` membaca semua berkas `*.txt`, mengurutkan namanya, lalu mengacak dengan `seed`. Sebelum token digabung, berkas dipisah menjadi train dan validasi. Untuk N berkas dan fraksi validasi r, kode menggunakan

$$N_{\text{valid}} = \max(1, \min(N - 1, \text{round}(Nr))), \qquad N_{\text{train}} = N - N_{\text{valid}}.$$

Sebagai contoh, $N = 60$ dan $r = 0{,}25$ menghasilkan 15 cerita validasi dan 45 cerita train. Pemisahan per berkas mencegah awal dan akhir cerita yang sama masuk ke kedua kelompok. Setelah dibagi, setiap kelompok digabung menjadi satu aliran token, dengan ID 256 di antara cerita. Karena itu satu jendela dapat melintasi batas dua cerita; batasnya terlihat oleh model melalui ID 256.

Analogi. Jangan membagi halaman satu buku ke dua ruang ujian. Simpan seluruh buku di ruang train atau seluruhnya di ruang validasi. Sesudah pembagian, halaman-halaman dalam tiap ruang baru disusun menjadi tumpukan.

2.3 Pasangan input–target: `sample_batch`

Misalkan aliran token adalah $C = (c_0, c_1, \ldots, c_{N-1})$. Untuk panjang konteks L, sebuah titik awal s menghasilkan

$$X = (c_s, c_{s+1}, \ldots, c_{s+L-1}), \qquad Y = (c_{s+1}, c_{s+2}, \ldots, c_{s+L}).$$

Target Y bergeser satu posisi dari input X. Jika alirannya $[10, 20, 30, 40, 50]$ dan $L = 3$, salah satu pasangan adalah $X = [10, 20, 30]$ dan $Y = [20, 30, 40]$. Model belajar bahwa setelah 10 muncul 20, setelah 20 muncul 30, dan seterusnya. `batch_size=B` berarti ada B jendela seperti ini dalam satu microbatch. Bentuk tensor

$$X, Y \in \mathbb{Z}^{B \times L}.$$

Kode memilih s secara acak dari aliran yang sesuai dan memindahkan batch ke `device`.

2.4 Konfigurasi: `config.py` dan tiga JSON

`ModelConfig` menyimpan L (`context_length`), d (`n_embed`), jumlah head H, dan jumlah blok K. `TrainConfig` menyimpan data, output, seed, batch, langkah, dan pengaturan optimizer. `SparseConfig` menyimpan pilihan untuk tiap varian MLP. `load_config` membaca JSON dan membangun tiga kelompok pengaturan itu. Nilai yang tidak sah, misalnya d tidak habis dibagi H, ditolak sebelum training.

Konfigurasi	Kegunaan
`configs/smoke.json`	Dua langkah per varian; memeriksa alur, bukan kualitas cerita.
`configs/small_m1.json`	Model $d = 192$, $H = 4$, $K = 4$, $L = 256$ pada 12 cerita demo.
`stories_expanded_m1.json`	60 cerita, fraksi validasi 0,25, 600 langkah; output terpisah.

Pada program lokal, `device="auto"` memilih MPS jika tersedia dan CPU jika tidak. Notebook Colab memilih CUDA jika GPU aktif, selain itu CPU. Mengubah ukuran model membuat checkpoint lama tidak cocok dengan arsitektur baru.

3 Langkah 2: Transformer membuat prediksi

3.1 Embedding token dan posisi

Untuk token t_i pada posisi i, model mengambil dua vektor berdimensi d:

$$\mathbf{x}_i = E_{\text{token}}[t_i] + E_{\text{pos}}[i], \quad E_{\text{token}} \in \mathbb{R}^{257 \times d}, \quad E_{\text{pos}} \in \mathbb{R}^{L \times d}.$$

Penjumlahan ini dilakukan oleh `token_embedding` dan `position_embedding`. Vektor token memberi identitas potongan teks; vektor posisi memberi tahu di mana ia berada dalam jendela. Bentuk hasilnya $B \times L \times d$.

Analogi. Kartu bertuliskan "pagi" memberi isi, nomor kursi memberi posisi. Dua informasi itu diperlukan agar murid dapat membedakan "pagi itu" dari "itu pagi".

3.2 Causal attention: `CausalAttention`

Setiap head memproyeksikan vektor menjadi query, key, dan value:

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V, \qquad d_h = d/H.$$

Untuk posisi i yang ingin memprediksi token berikutnya, skor melihat posisi j adalah

$$a_{ij} = \frac{\mathbf{q}_i^\top \mathbf{k}_j}{\sqrt{d_h}} + M_{ij}, \qquad M_{ij} = \begin{cases} 0, & j \le i, \\ -\infty, & j > i. \end{cases}$$

Softmax mengubah skor menjadi bobot perhatian $\alpha_{ij} = \exp(a_{ij}) / \sum_{k \le i} \exp(a_{ik})$. Keluaran head pada posisi i adalah $\mathbf{o}_i = \sum_{j \le i} \alpha_{ij} \mathbf{v}_j$. Keluaran semua head digabung dan diproyeksikan lagi.

Analogi. Query adalah pertanyaan yang diajukan kata saat ini; key adalah label pada catatan lama; value adalah isi catatan. Aturan *causal* menutup halaman-halaman masa depan. Tanpanya, model bisa mengintip jawaban saat latihan.

Perhatikan. Kode lokal membuat mask segitiga bawah dengan `torch.tril`. Notebook Colab memakai operasi `scaled_dot_product_attention(..., is_causal=True)`; tujuan matematisnya sama. Untuk $d = 192$ dan $H = 4$, tiap head memakai $d_h = 48$ dimensi.

3.3 Residual, LayerNorm, dan MLP

Satu `Block` menjalankan dua operasi berurutan:

$$A = X + \text{Attention}(\text{LN}(X)), \tag{1}$$

$$X_{\text{next}} = A + \text{MLP}(\text{LN}(A)). \tag{2}$$

Tanda + adalah *residual connection*: informasi lama tetap punya jalur langsung. `LayerNorm` menstabilkan skala vektor. MLP dense mengembangkan lebar dari d ke $4d$, menerapkan ReLU, lalu kembali ke d:

$$\text{MLP}(\mathbf{x}) = W_2 \, \text{ReLU}(W_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2, \quad W_1 \in \mathbb{R}^{4d \times d}, \quad W_2 \in \mathbb{R}^{d \times 4d}.$$

Untuk $d = 192$, lebar tersembunyi MLP adalah 768. Blok diulang $K = 4$ kali pada konfigurasi M1.

Analogi. Attention mengumpulkan catatan yang relevan dari bagian cerita sebelumnya. MLP memikirkan ulang catatan itu untuk posisi saat ini. Residual seperti menyimpan salinan catatan asli agar tidak hilang saat ditafsirkan.

3.4 Logit, probabilitas, dan prediksi berikutnya

Setelah blok terakhir, `final_norm` dan `lm_head` menghasilkan logit $z_{i,v}$ untuk setiap $v \in \mathcal{V}$:

$$Z = \text{Linear}(\text{LN}(X_{\text{akhir}})), \qquad Z \in \mathbb{R}^{B \times L \times 257}.$$

Probabilitas token v pada posisi i diperoleh melalui softmax,

$$p(v \mid t_{\le i}) = \frac{\exp(z_{i,v})}{\sum_{u=0}^{256} \exp(z_{i,u})}.$$

Logit besar berarti peluang besar relatif terhadap token lain; logit sendiri bukan persentase.

4 Langkah 3: lima pilihan MLP dalam `sparse.py`

Semua varian memakai tokenizer, attention, data, dan prosedur training yang sama. Setelah Transformer dense dibuat, `convert_model` mengganti MLP pada tiap blok.

4.1 Dense: acuan

Dense memakai seluruh $4d$ neuron dan seluruh bobot W_1, W_2. Nilai loss, kecepatan, dan memori dense menjadi acuan untuk empat varian lain.

Analogi. Semua pekerja di ruang kerja mendapat tugas pada setiap token.

4.2 Pruned: mask bobot tetap

Kelas `PrunedMLP` mencari bobot dengan nilai mutlak terkecil saat model dibentuk. Untuk target sparsity s, sekitar fraksi s elemen pada tiap matriks diberi mask nol. Operasi yang dipakai saat `forward`:

$$W_{\text{efektif}} = W \odot M, \qquad M_{ij} \in \{0, 1\}.$$

Simbol $\odot$ berarti perkalian elemen demi elemen. Posisi dengan $M_{ij} = 0$ tidak berpengaruh pada keluaran. Mask disimpan sebagai buffer, bukan parameter yang dilatih. Bobot matriks tetap dialokasikan penuh; implementasi ini tidak menjamin RAM atau waktu lebih kecil.

Analogi. Sebuah peta jalan tetap dicetak lengkap, tetapi beberapa ruas diberi tanda tertutup. Menutup ruas bukan berarti ukuran kertas petanya mengecil.

4.3 TopK: beberapa neuron dipakai per token

`TopKMLP` terlebih dahulu menghitung $\mathbf{h} = \text{ReLU}(W_1\mathbf{x} + \mathbf{b}_1)$. Dari $4d$ aktivasi, hanya $k = \max(1, \text{round}(4df))$ yang terbesar dipilih, dengan $f =$ `active_fraction`. Keluaran:

$$\mathbf{y} = \mathbf{b}_2 + \sum_{j \in \text{TopK}(\mathbf{h}, k)} h_j W_{2,:,j}.$$

Dengan $d = 192$ dan $f = 0{,}25$, $4d = 768$ dan $k = 192$ neuron dipakai untuk setiap token. Lapisan pertama tetap menghitung seluruh 768 aktivasi. Pemilihan `topk` dan pengumpulan kolom dapat memiliki biaya sendiri; angka neuron aktif tidak otomatis berarti akselerasi empat kali.

Analogi. Semua pekerja mengajukan ide awal, lalu hanya ide dengan skor tertinggi dibawa ke rapat akhir. Biaya mengumpulkan ide awal masih ada.

4.4 MoE: memilih beberapa expert

`MoEMLP` membagi lebar MLP dense ke E expert. Router atau *gate* memberi logit g_e untuk expert e dan memilih himpunan $S = \text{TopK}(\mathbf{g}, k)$:

$$\alpha_e = \frac{\exp(g_e)}{\sum_{j \in S} \exp(g_j)} \quad (e \in S), \qquad \mathbf{y} = k \sum_{e \in S} \alpha_e \, \text{Expert}_e(\mathbf{x}).$$

Faktor k berasal dari implementasi ini untuk menjaga skala ketika beberapa expert dijumlahkan. Nilai `top_k=2` dan `n_experts=4` berarti dua dari empat expert dipakai per token. Expert mula-mula menerima potongan bobot MLP dense, sementara router dilatih dari awal.

Tanpa penyeimbang, router dapat terlalu sering memilih expert yang sama. Kode menambahkan loss bantu:

$$\mathcal{L}_{\text{aux}} = E \sum_{e=1}^{E} I_e L_e, \qquad I_e = \text{mean}(\text{softmax}(\mathbf{g})_e), \quad L_e = \text{mean}(\mathbf{1}[e \in S]).$$

Loss total saat training ialah $\mathcal{L}_{\text{token}} + \lambda \mathcal{L}_{\text{aux}}$. Nilai λ diatur oleh `aux_coef`. Metrik `train_loss` dan `valid_loss` hanya melaporkan loss token agar prediksi dapat dibandingkan.

Analogi. Router adalah petugas penerima naskah. Ia mengirim satu potongan cerita ke dua editor dari empat editor yang tersedia. Loss bantu mencegah satu editor terus-menerus kebanjiran pekerjaan.

4.5 SET: mask berubah selama training

`SETMLP` membuat mask acak dengan kerapatan awal $\rho =$ `density`, sehingga sekitar ρ fraksi koneksi aktif. Selama training, bobot bermask nol dipaksa tetap nol. Setiap `rewire_every` langkah, `rewire_set` melakukan:

1. Cari koneksi aktif dengan $|w|$ terkecil dan matikan sejumlah $r = \text{round}(\texttt{rewire_fraction} \times N_{\text{aktif}})$.
2. Pilih r posisi tidak aktif secara acak dan hidupkan.
3. Nolkan bobot pada posisi yang berubah dan state optimizer yang bentuknya sama dengan matriks bobot.

Jumlah koneksi aktif tetap, tetapi lokasinya berubah. Menolkan state optimizer penting karena momentum lama tidak boleh terbawa ke sambungan baru.

Analogi. Jumlah kursi dalam tim tetap. Secara berkala kursi dengan kontribusi terendah ditutup dan kursi baru dibuka di tempat lain. Catatan kerja lama pada kursi yang berpindah dihapus.

Perhatikan. `active_connections` menghitung elemen mask aktif pada `pruned` dan `set`. Ukuran itu tidak sama dengan jumlah parameter fisik atau banyaknya operasi GPU untuk `topk` dan `moe`.

5 Langkah 4: kesalahan mengubah bobot

5.1 Cross-entropy: `model.py`

Untuk target benar $y_{b,i}$ pada batch b, posisi i, model memperoleh probabilitas $p(y_{b,i} \mid X_{b,\le i})$. Loss rata-rata:

$$\mathcal{L}_{\text{token}} = -\frac{1}{BL} \sum_{b=1}^{B} \sum_{i=1}^{L} \log p(y_{b,i} \mid X_{b,\le i}).$$

Contoh satu tebakan: jika token benar diberi peluang 0,5, kesalahannya $-\log(0{,}5) \approx 0{,}693$. Jika peluangnya hanya 0,01, kesalahannya $-\log(0{,}01) \approx 4{,}605$. Kode memakai `F.cross_entropy` pada logit yang diratakan ke bentuk $(BL) \times 257$ dan target sebanyak BL.

Jika seluruh 257 token diberi peluang sama, loss-nya $\log(257) \approx 5{,}55$. Itu acuan kasar untuk tebakan seragam, **bukan** target mutu cerita. Loss rendah juga bisa terjadi karena model menguasai ejaan dan tanda baca tanpa memahami alur.

Analogi. Guru menghukum tebakan yang yakin tetapi salah lebih besar daripada tebakan yang masih memberi peluang wajar pada jawaban benar.

5.2 Akumulasi gradien dan optimizer: `train.py`

Satu langkah optimizer terdiri atas $G =$ `grad_accum` microbatch. Kode membagi loss tiap microbatch dengan G sebelum `backward`. Maka gradien yang terkumpul setara dengan rata-ratanya:

$$\mathbf{g} = \frac{1}{G} \sum_{j=1}^{G} \nabla_\theta \mathcal{L}_j.$$

Jumlah posisi token yang diproses per langkah adalah BGL. Pada konfigurasi M1: $B = 2$, $G = 4$, $L = 256$, sehingga $BGL = 2048$ posisi per langkah. Dalam 300 langkah, model memproses 614 400 posisi, tetapi banyak jendela diambil berulang dari korpus yang jauh lebih kecil. Jumlah posisi diproses tidak sama dengan jumlah byte unik.

Setelah `backward`, kode memotong norma gradien pada `grad_clip=c`:

$$\mathbf{g}_{\text{clip}} = \mathbf{g} \min\left(1, \frac{c}{\|\mathbf{g}\|_2}\right).$$

Kemudian `AdamW` memakai riwayat gradien untuk memperbarui parameter. Bentuk ringkas satu langkahnya:

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1-\beta_1)\mathbf{g}_t, \tag{3}$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1-\beta_2)\mathbf{g}_t^2, \tag{4}$$

$$\theta_{t+1} \approx (1 - \eta\lambda_{\text{wd}})\theta_t - \eta \frac{\widehat{\mathbf{m}}_t}{\sqrt{\widehat{\mathbf{v}}_t} + \epsilon}. \tag{5}$$

Di sini η adalah `learning_rate`; λ_{wd} adalah `weight_decay`. Implementasi rinci, termasuk koreksi bias, ditangani PyTorch.

Analogi. Gradien adalah petunjuk arah turun bukit. Akumulasi meminta pendapat beberapa kelompok kecil sebelum melangkah. Gradient clipping mencegah satu pendapat ekstrem membuat langkah terlalu jauh. AdamW mengingat arah yang sering membantu.

5.3 Urutan satu langkah dalam kode

1. `model.train()` mengaktifkan mode training dan `optimizer.zero_grad()` mengosongkan gradien lama.
2. Ambil G batch. Untuk setiap batch: hitung logit, loss token, tambahkan loss bantu MoE bila perlu, bagi dengan G, lalu `backward()`.
3. Periksa bahwa loss berupa angka hingga; potong gradien; panggil `optimizer.step()`.
4. Jika varian SET, paksa bobot tidak aktif ke nol dan jalankan rewiring pada intervalnya.
5. Tulis `metrics.jsonl`: catat waktu langkah dan, pada interval tertentu, evaluasi pada data validasi.

Perhatikan. Pada MPS, `_sync` menunggu operasi perangkat selesai sebelum dan sesudah pengukuran waktu. Tanpa sinkronisasi, waktu dapat terlihat lebih kecil karena tugas GPU masih berjalan.

6 Langkah 5: evaluasi dan berkas hasil

6.1 Evaluasi lokal: `evaluate` dalam `train.py`

Model masuk ke mode `eval` dan berjalan tanpa gradien. Fungsi ini mengambil `eval_batches` jendela acak dari cerita validasi, kemudian menghitung rata-rata cross-entropy:

$$\mathcal{L}_{\text{valid,lokal}} = \frac{1}{J}\sum_{j=1}^{J} \mathcal{L}_j, \qquad J = \texttt{eval_batches}.$$

Generator titik awal selalu dibuat lagi dengan `seed + 2`. Jadi, pada setiap evaluasi, jendela validasi yang dipakai sama. Perubahan loss lebih mudah ditafsirkan karena pertanyaannya tidak berganti-ganti. Namun yang diukur tetap sampel jendela, bukan semua byte validasi.

Analogi. Murid mengerjakan set soal ujian yang sama setiap beberapa jam. Kemajuannya terlihat lebih jelas daripada bila soalnya selalu diacak, tetapi hasil itu belum mewakili semua kemungkinan soal.

Baris langkah 0 menyimpan loss validasi sebelum pembaruan bobot. `eval_every` menentukan interval evaluasi berikutnya; langkah terakhir selalu dievaluasi. `log_every` menentukan kapan baris training dicatat. Pada baris tanpa evaluasi, `valid_loss` bernilai `null`, bukan nol.

6.2 Cara membaca hasil

`metrics.jsonl` berisi satu objek JSON per baris, misalnya:

```
{"step": 100, "train_loss": 2.20, "valid_loss": 2.30,
 "step_seconds": 0.06}
```

`summary.json` meringkas nilai terakhir, jumlah parameter, pembagian berkas, jumlah token, waktu, dan memori. `model.pt` berisi bobot, konfigurasi arsitektur, konfigurasi sparsity, dan nama varian. Checkpoint lokal tidak menyimpan state AdamW atau posisi generator acak; menjalankan training lagi berarti memulai percobaan baru.

Metrik	Cara membaca
`train_loss`	Loss token pada microbatch langkah terakhir; berfluktuasi karena batch berubah.
`valid_loss`	Loss pada jendela validasi yang dipilih dengan seed tetap; bandingkan dalam eksperimen yang datanya sama.
`initial_valid_loss`	Titik awal sebelum training.
`train_tokens_per_second`	$BGL \times$ `steps` dibagi waktu langkah training; evaluasi dan generasi tidak dihitung.
`peak_process_rss_mb`	Puncak memori proses Python. Ini bukan ukuran lengkap semua memori GPU atau biaya produksi.

Jika `train_loss` turun tetapi `valid_loss` naik terus, model mungkin terlalu menyesuaikan diri pada data train. Jika keduanya turun, model belajar memprediksi token pada distribusi yang diuji. Kualitas cerita masih harus diperiksa dengan membaca contoh baru.

Perplexity per token dapat dihitung dari cross-entropy:

$$\mathrm{PPL} = \exp(\mathcal{L}).$$

Karena token di sini adalah byte, ini adalah perplexity per byte. Jangan bandingkan langsung dengan angka dari tokenizer kata atau subkata. Program lokal tidak menyimpan perplexity sebagai kolom terpisah.

6.3 Perbandingan: `compare.py`

`compare.py` menjalankan dense dan empat varian sparse dengan konfigurasi yang sama. Secara bawaan tiap varian memakai proses Python terpisah, agar puncak RSS satu varian tidak terbawa ke pengukuran varian lain. Setelah itu program mengumpulkan `summary.json` menjadi `comparison.csv`. Opsi `-variants` memilih sebagian varian; `-overwrite` diperlukan jika folder output varian sudah ada dan pengguna memang ingin menggantinya. Dengan `-in-process`, varian dijalankan dalam satu proses dan angka puncak RSS per varian dikosongkan.

Perhatikan. Perbandingan adil memerlukan data, pembagian train/validasi, seed, batch token, jumlah langkah, optimizer, dan perangkat yang sama. Jumlah parameter serta operasi tiap varian tetap berbeda. Loss dari `stories_demo` tidak langsung sebanding dengan loss dari `stories_expanded` atau dataset Frog karena data ujinya berubah.

7 Langkah 6: model menulis lanjutan

`generate.py` memuat `model.pt` dengan `torch.load(..., weights_only=True)`, membangun kembali Transformer dan jenis MLP dari metadata, lalu memuat `state_dict`. Prompt diubah menjadi byte, kemudian `StoryTransformer.generate` mengulang:

1. Ambil paling banyak L token terakhir sebagai konteks.

2. Hitung logit pada posisi terakhir.

3. Bagi logit dengan suhu $T > 0$, ubah menjadi probabilitas, pilih satu token, lalu tambahkan ke konteks.

4. Berhenti jika ID 256 terpilih atau batas token baru tercapai.

Distribusi sampling pada suhu T adalah

$$p_T(v) = \frac{\exp(z_v/T)}{\sum_{u \in A} \exp(z_u/T)}, \qquad v \in A,$$

dengan A himpunan token yang diizinkan. Suhu lebih rendah membuat pilihan lebih terkonsentrasi pada logit tinggi; suhu lebih tinggi membuat hasil lebih bervariasi. Mengubah suhu tidak mengajari model fakta atau alur baru.

7.1 Mengapa keluaran tetap UTF-8 tetapi bisa aneh?

Karena satu karakter Unicode dapat memakai beberapa byte, sampling byte secara bebas bisa menghasilkan urutan yang rusak. Implementasi lokal melacak apakah model sedang berada di tengah karakter UTF-8. Jika ya, hanya byte lanjutan yang sah diizinkan. Di akhir batas token, model tidak boleh memulai karakter yang tidak akan sempat diselesaikan.

Analogi. Untuk menulis satu huruf yang terdiri dari tiga potong puzzle, model harus menyelesaikan potong kedua dan ketiga sebelum memulai huruf baru. Aturan ini menjaga bentuk tulisannya, bukan isi ceritanya.

Generasi menampilkan kembali prompt diikuti kelanjutannya. Batas `-max-new-tokens` menghitung byte baru, bukan kata atau kalimat. Model 300 langkah pada 12 cerita dapat membuat suku kata yang menyerupai bahasa Indonesia tetapi belum menghasilkan cerita runtut.

8 Tes: apa yang dibuktikan `test_small_llm.py`?

1. **Tokenizer dan data:** teks UTF-8 dapat dikodekan dan dibaca kembali; berkas train dan validasi tidak tumpang tindih; target adalah input yang bergeser satu token.

2. **Causal attention:** mengubah token masa depan tidak mengubah logit posisi sebelumnya.

3. **TopK dan MoE:** dalam pengaturan yang memakai semua neuron/expert, keluaran dapat disamakan dengan MLP dense sesuai konstruksi tes.

4. **SET:** rewiring menjaga jumlah koneksi aktif dan menghapus state optimizer pada posisi yang berubah.

5. **Checkpoint:** arsitektur sparse dapat dibangun kembali dan menghasilkan keluaran yang sama sesudah bobot dimuat.

6. **Training singkat:** setiap varian dapat menjalankan satu langkah dan menyimpan checkpoint dengan loss hingga.

7. **Generasi:** byte baru mengikuti urutan UTF-8 yang valid pada contoh yang sengaja memberi skor tinggi pada byte tidak sah.

Perhatikan. Tes ini membuktikan sifat kode pada contoh kecil. Tes satu langkah tidak membuktikan bahwa model mampu bercerita. Kualitas bahasa memerlukan data dan evaluasi yang terpisah.

9 Versi mandiri Colab: apa yang sama dan berbeda?

Notebook `small_llm_frog_storytelling_colab.ipynb` menulis ulang tokenizer, Transformer, kelima varian MLP, training, checkpoint, dan generasi di dalam satu berkas. Ia tidak mengimpor `small_llm/*.py` atau cerita lokal. Data dibaca dari halaman dataset Frog Storytelling di Hugging Face. Dataset tersebut mewajibkan persetujuan akses di situs Hugging Face dan login akun yang sama di Colab. Notebook memeriksa apakah kolom `text` ada dan menampilkan nama kolom jika tidak ada.

1. Sel pemasangan dependensi dan login menyiapkan pembacaan Parquet dari alamat `hf://`. Token akses tidak ditulis dalam notebook.
2. Sel data membersihkan spasi, membuang baris kosong dan duplikat identik, lalu membagi dokumen train/validasi dengan seed tetap. Dokumen baru digabung menjadi aliran byte setelah pembagian.
3. Sel model dan varian menerapkan rumus pada Bagian **??**–**??**. Attention Colab memakai operasi PyTorch `is_causal=True`.
4. Sel training melatih `dense` secara bawaan. `RUN_COMPARISON` perlu diubah menjadi `True` untuk melatih empat varian lain.
5. Sel evaluasi membaca semua byte validasi secara berurutan dalam potongan konteks, bukan beberapa jendela acak seperti `train.py` lokal. Ia menyimpan loss validasi, perplexity per byte, dan akurasi prediksi byte.
6. Saat loss validasi membaik, notebook menyimpan `best_model.pt`. Model untuk generasi dimuat dari checkpoint terbaik ini, bukan otomatis dari langkah terakhir.
7. Generasi notebook membatasi karakter pada karakter yang muncul dalam dokumen train melalui trie byte UTF-8. Aturan ini lebih ketat daripada mask UTF-8 umum pada program lokal, tetapi tetap tidak menjamin alur cerita masuk akal.
8. Sel terakhir mengemas hasil ke ZIP agar dapat diunduh sebelum runtime Colab berhenti.

Perhatikan. Dataset Frog berasal dari penceritaan atas gambar yang sama. Dokumen validasi dipisah, tetapi topik dan urutan adegan antar cerita dapat sangat mirip. Angka validasi karenanya tidak boleh dianggap bukti kemampuan menulis cerita bertopik bebas. Nilai notebook juga tidak langsung sebanding dengan nilai program lokal: dataset, cara evaluasi, jumlah langkah, dan checkpoint yang dipilih berbeda.

10 Contoh ukuran tensor dari awal sampai akhir

Dengan konfigurasi `small_m1.json`, ambil $B = 2$, $L = 256$, $d = 192$, $H = 4$, dan $V = 257$. Ikuti satu batch:

Tahap	Bentuk tensor
Input dan target	2×256
Embedding token + posisi	$2 \times 256 \times 192$
Query/key/value sebelum dipisah head	masing-masing $2 \times 256 \times 192$
Query/key/value sesudah dipisah head	masing-masing $2 \times 4 \times 256 \times 48$
Skor attention per head	$2 \times 4 \times 256 \times 256$
Aktivasi tersembunyi MLP dense	$2 \times 256 \times 768$
Logit keluaran	$2 \times 256 \times 257$
Target untuk cross-entropy	512 ID setelah diratakan

Tabel ini berguna saat mengubah arsitektur. Jika sebuah operasi gagal karena ukuran tensor, cari baris pertama yang bentuknya tidak sesuai.

11 Langkah pengembangan yang dapat diulang

11.1 Mulai dari satu pertanyaan

Contoh pertanyaan: apakah konteks lebih panjang membantu loss validasi dan kesinambungan kalimat? Buat satu baseline, simpan konfigurasi, seed, pembagian data, `metrics.jsonl`, dan beberapa prompt yang tetap. Pada percobaan berikutnya, ubah satu hal, misalnya `context_length`. Jika sekaligus mengubah dataset dan tokenizer, sulit mengetahui penyebab perubahan hasil.

Ide	Perubahan kode/konfigurasi	Pemeriksaan yang diperlukan
Tambah data	Ganti `data_dir`; bagi per cerita	Periksa duplikat dan jumlah cerita validasi; jangan bandingkan loss mentah dengan dataset lama.
Konteks lebih panjang	Naikkan `context_length`	Periksa RAM, bentuk mask/posisi, loss, dan kelanjutan cerita.
Tokenizer subkata	Ganti `encode/decode`, ukuran kosakata, embedding, dan head keluaran	Latih ulang dari awal; checkpoint tokenizer byte tidak kompatibel.
MLP sparse	Ubah satu opsi sparsity/top-k/moe/set	Catat loss, jumlah parameter, koneksi aktif, waktu, dan memori.
Berhenti lebih dini	Simpan checkpoint terbaik menurut `valid_loss`	Pastikan validasi tidak dipakai untuk melatih bobot; sediakan set uji terpisah untuk klaim akhir.

11.2 Urutan praktik yang disarankan

1. **Periksa data.** Hitung cerita, panjang byte, duplikat, dan pembagian train/validasi. Baca beberapa contoh. Pastikan validasi berisi cerita utuh yang tidak ada di train.

2. **Jalankan smoke hanya untuk memeriksa kode.** Dua langkah tidak cukup untuk menilai bahasa.

3. **Latih dense lebih dulu.** Simpan kurva loss, ringkasan, dan checkpoint. Gunakan prompt dan suhu yang tetap saat membandingkan contoh generasi.

4. **Periksa generalisasi.** Lihat apakah loss validasi masih turun. Jangan menyimpulkan kemampuan bercerita dari loss saja; baca apakah tokoh, peristiwa, dan akhir cerita saling berhubungan.

5. **Ubah satu variabel.** Ulangi beberapa seed jika biaya memungkinkan. Varian sparse harus dibandingkan pada data dan anggaran training yang sama.

6. **Simpan catatan.** Tulis hipotesis, konfigurasi, metrik, dan contoh keluaran. Hasil yang gagal juga berguna jika penyebabnya jelas.

Analogi. Mengembangkan model seperti mencoba resep. Jika Anda mengganti tepung, suhu oven, dan waktu panggang sekaligus, Anda tidak tahu perubahan mana yang membuat hasil lebih baik atau buruk.

11.3 Tiga batas yang sering menipu

1. **Data kecil.** Model dapat menghafal susunan suku kata dan gaya tanpa belajar menyusun cerita baru. Bahkan 60 cerita tambahan masih korpus yang sangat kecil untuk sekitar 1,9 juta parameter.

2. **Konteks byte pendek.** Jendela 256 byte sering hanya memuat sebagian paragraf. Model sulit menjaga hubungan jauh antara awal dan akhir cerita jika awalnya sudah keluar dari jendela.

3. **Metrik token bukan metrik cerita.** Loss dan perplexity mengukur prediksi token. Cerita yang runtut perlu pemeriksaan isi: apakah tokoh konsisten, sebab-akibat jelas, dan akhir terkait dengan awal.

12 Perintah dan keluaran yang perlu dikenal

Perintah berikut adalah contoh untuk dijalankan pengguna dari root repo. Dokumen ini sendiri tidak menjalankan training.

```
# Pemeriksaan kecil
.venv/bin/python -m unittest discover -s tests
.venv/bin/python -m small_llm.compare --config configs/smoke.json --overwrite

# Satu model dense dengan korpus tambahan
.venv/bin/python -m small_llm.train \
  --config configs/stories_expanded_m1.json --variant dense

# Lihat metrik dan coba checkpoint
cat artifacts/stories_expanded_m1/dense/summary.json
cat artifacts/stories_expanded_m1/dense/metrics.jsonl
.venv/bin/python -m small_llm.generate \
  --checkpoint artifacts/stories_expanded_m1/dense/model.pt \
  --prompt "Pagi itu, Mira menemukan "
```

Folder hasil yang sudah ada tidak ditimpa tanpa `-overwrite`. Opsi itu berarti mengganti hasil lama, sehingga sebaiknya simpan folder eksperimen yang ingin dipertahankan.

13 Ringkasan hubungan rumus dan kode

Konsep	Rumus inti	Lokasi kode
Byte dan EOS	$\mathcal{V} = \{0, \ldots, 256\}$	`tokenizer.py`: `encode`, `decode`
Batch bergeser	$Y_i = X_{i+1}$	`data.py`: `sample_batch`
Embedding	$\mathbf{x}_i = E_t[t_i] + E_p[i]$	`model.py`: `StoryTransformer.forward`
Attention causal	$\mathrm{softmax}(QK^\top/\sqrt{d_h} + M)V$	`model.py`: `CausalAttention`
MLP dense	$W_2\,\mathrm{ReLU}(W_1 x + b_1) + b_2$	`model.py`: `DenseMLP`
Loss token	$-\,\mathrm{mean}\log p(y)$	`model.py`: `F.cross_entropy`
Mask sparse	$W_{\text{efektif}} = W \odot M$	`sparse.py`: `PrunedMLP`, `SETMLP`
TopK neuron	$\sum_{j\in S} h_j W_{2,:,j} + b_2$	`sparse.py`: `TopKMLP`
MoE	$k \sum_{e\in S} \alpha_e E_e(x)$	`sparse.py`: `MoEMLP`
Akumulasi gradien	$G^{-1} \sum_j \nabla \mathcal{L}_j$	`train.py`: `train_variant`

Konsep	Rumus inti	Lokasi kode
Sampling suhu	softmax(z/T)	`model.py`: `generate`

Akhir panduan. Untuk mengembangkan model, mulailah dari peta pada Bagian **??**, ikuti satu batch melalui Bagian **??** dan **??**, lalu gunakan Bagian **??** untuk merancang satu percobaan yang dapat diperiksa kembali.